

ULAL

Unified Lifelong Action Learning

Complete Pipeline Guide
with every design question answered

ResNet50 -> LSTM -> head
(23.5M) (2.36M) (70K)

THE FREEZE LINE MOVES. That is the whole project.

"How cheap can memory get before the model gets stupid?"

Video action recognition on UCF101 • Continual Learning • Knowledge Distillation •
Generative Replay • Meta-Learning • Active Domain Adaptation

Contents

Part	Topic	Page
1	The one idea: the freeze line moves	2
2	The model: three parts, layer by layer	2
3	The pipeline, step by step (10 + 3 + 3 example)	4
4	The complete schema	8
5	The six continual-learning methods	9
6	Sections 5-10: what comes next	11
7	Every question you asked, answered	12
8	Repository layout and run order	20
9	Bugs found and fixed	21

Part 7 in detail — the questions

Q	Question	Page
Q1	Does the GAN part make sense in this pipeline?	12
Q2	Do we train just the head or all of it? Does the project make sense?	12
Q3	Is it better with cached embeddings or with real frames?	14
Q4	Are sections 5, 7, 8 alternatives -- do I choose one?	14
Q5	Why does Task1 forget so much after Task2?	16
Q6	Does KD inside rehearsal make sense, passing the same model?	16
Q7	ResNet was fine-tuned on 10 classes -- are the new embeddings wrong?	18

Part 1 — The one idea

The model has three parts. **The freeze line moves.** That is the entire project.

ResNet50 (23.5M)	->	LSTM (2.36M)	->	head (70K)
<- move line LEFT		move line RIGHT ->		
more trains		less trains		
representation adapts		cannot adapt		
more memory needed		cheaper		

The research question: How much of a video model must stay trainable for lifelong learning, and what does it cost in memory?

Every topic in this project is one point on that curve. Nothing is decoration.

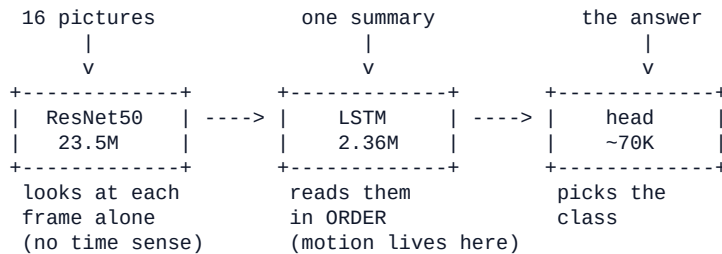
The trade-off table (this is the thesis)

Freeze boundary	Trainable	Replay unit	Memory/class	Adapts?
Nothing frozen	25.9M	raw clip	144 MB	fully
After layer4	2.43M	16 x 2048	~2 MB	YES

After LSTM	70K	256-d	15 KB	no
After LSTM + GAN	70K	(mu, sigma)	2 KB	no

Five orders of magnitude of memory. That span is the contribution.

Part 2 — The model: three parts



Part	Question it answers	Example
ResNet50	What is in this picture?	person, water, splash
LSTM	What is happening across the 16?	body up -> rotate -> enter water
head	Which of my classes is that?	Diving

Layer by layer (verified numbers)

conv1	9,408	FROZEN	+	
bn1	128	FROZEN		
layer1	215,808	FROZEN		8,543,296 keep ImageNet knowledge
layer2	1,219,584	FROZEN		(edges, textures, shapes -- generic)
layer3	7,098,368	FROZEN	+	

layer4	14,964,736	TRAINS	+	
LSTM	2,361,344	TRAINS		17,328,650 learn YOUR classes = 67%
fc	2,570	TRAINS	+	

Why freeze the early layers? They detect edges and textures — the same for cats, cars and diving. No reason to relearn them. Only the last block needs to become about your classes. The ImageNet fc (2048 -> 1000) is discarded entirely.

Part 3 — The pipeline, step by step

Worked example: **10 base classes, then +3, then +3.**

```
BASE (10)  PlayingTabla, PommelHorse, JumpingJack, PushUps, PoleVault,
           HorseRace, HighJump, Drumming, HorseRiding, Diving
           <-- all full-body sports

TASK 1 (3)  ApplyEyeMakeup, ApplyLipstick, Archery
           <-- close-up faces

TASK 2 (3)  BoxingPunchingBag, BoxingSpeedBag, BrushingTeeth
           <-- fine-grained + one more face
```

Remember these groupings. They explain everything that happens later. Base is full-body sport; Task1 and part of Task2 are close-up faces with a hand near the mouth. Those will compete for the same region of feature space.

Step 1 — A video becomes a clip

```
Diving_01.avi  (3 seconds, ~75 frames)
|
|  take 16 frames, evenly spread
|  resize to 256, crop centre 224x224
|  normalise colours (ImageNet statistics)
v
Diving_01.pt   (16, 3, 224, 224) = 9.63 MB
```

Choice	Why
16 frames	One photo cannot tell diving from falling. You need motion. 16 is enough to see it, few enough to fit in memory.
224 x 224	That is the input size ResNet50 was designed for.
ImageNet normalise	The backbone was pretrained with those statistics.

Step 2 — Section 1: fine-tune on the 10 base classes

You tried five backbones and ResNet50 won. Then you trained it — but not all of it. Only **layer4 + LSTM + fc** train: 17.3M parameters, 67% of the model.

Result: ResNet50_10C.pth, about 98.8% on the 10 base classes.

This is the only time 17M parameters ever train. Everything after this point is much smaller and much cheaper.

Step 3 — The cut

After Section 1 the whole ResNet is frozen forever. It will never learn again. That means it gives the **same answer for the same clip, every single time**:

```
Diving_01 -> ResNet -> [0.3, 1.7, 0.02, ...]    epoch 1
Diving_01 -> ResNet -> [0.3, 1.7, 0.02, ...]    epoch 2    identical
Diving_01 -> ResNet -> [0.3, 1.7, 0.02, ...]    epoch 15    identical
```

Recomputing that 15 times is pure waste. Compute ONCE and save.

```
conv1 - layer1 - layer2 - layer3 - layer4 +- LSTM - head
+----- frozen, run ONCE -----+
                                cut here
```

Step 4 — Section 2: extract the embeddings

Run the frozen ResNet over every clip once. Follow one batch of 8 clips:

```

(8, 16, 3, 224, 224)    8 clips x 16 frames
    | flatten clips and frames together
    v                    (the ResNet sees photos, not clips)
(128, 3, 224, 224)      128 separate pictures
    | frozen ResNet
    v
(128, 2048, 1, 1)        each picture -> 2048 numbers
    | fold back into clips
    v
(8, 16, 2048)            *** SAVE THIS ***

```

Format	Size per clip	Note
raw clip (fp32)	9.63 MB	cannot cache -- too big
frame features (fp16)	65.5 KB	147x smaller -- what we save
h_last (fp32)	1.0 KB	too lossy -- see below

Why this matters so much

	Time per epoch
without cache	~80 s (ResNet over 15,904 frames)
with cache	~2 s (LSTM only)

40x faster. That is what makes Section 5's twelve ablation runs a coffee break instead of an overnight job.

Why save at 2048 and not 256?

The obvious idea: save the LSTM's final 256 numbers instead — 65x smaller again. **Do not.** Those 256 numbers come *out of* the LSTM. If you save those, the LSTM can never change again: it is stuck knowing only sports, and can never learn what makeup looks like. Saving at 2048 (before the LSTM) keeps it free to learn.

Step 5 — Section 2b: are the embeddings any good?

Before training anything, a **probe** answers the key question in seconds. Mean-pool each clip from (16, 2048) to (2048,) and ask a dumb classifier to separate the classes.

Probe	What it measures
1-NN	pure geometry -- are same-class clips nearest neighbours? No training.
linear	how much linearly separable information the features contain
silhouette	how tight the clusters are versus how far apart
centroid distance	which classes sit close -- these will compete later

Group	classes	chance	1-NN	linear	vs chance
Base	10	10.0%	...	...	...
Task1	3	33.3%	...	...	...
Task2	3	33.3%	...	...	...
JOINT	16	6.3%	...	...	... <-- all together

JOINT linear probe	Meaning	Action
above 70%	features are good; representation is not the bottleneck	continue
40 - 70%	usable but strained; ceiling will sit near here	continue, note it

below 40%	features are weak for these classes	STOP -- fix classes first
-----------	-------------------------------------	---------------------------

The probe is a **30-second preview of the Section 3 ceiling**. If the joint probe reads 70%, do not expect any continual-learning method to beat that.

Step 6 — Section 3: where should the cut go?

The ResNet is definitely frozen. But **should the LSTM be frozen too?** Test both, training on all 16 classes at once — no tasks, no forgetting, just *how good can this possibly get*.

```
PROBE A    [16x2048] -> LSTM [FROZEN] -> head [TRAINS]      70,416 train
PROBE B    [16x2048] -> LSTM [TRAINS] -> head [TRAINS]     2,431,760 train
```

Both load the SAME Task-0 LSTM weights.
The only difference is requires_grad.

Result	Meaning	Use
B much greater than A	the LSTM was the bottleneck	B -- replay ~2 MB/class
B about equal to A	frozen features already suffice	A -- 34x cheaper

The second output matters more: THE CEILING

```
"SmartReplay = 71%"          <- means nothing
"SmartReplay = 71%, ceiling = 85%" <- means 71 out of a possible 85
```

Without a ceiling you cannot tell whether a number is good or terrible. Every result in Sections 4 to 10 is read against it. It also **separates two failure modes**: if joint training reaches 85% with the same frozen features, then the features provably can hold all the classes, and any lower score is *forgetting* rather than a bad representation.

Step 7 — Section 4: the forgetting problem

```

You have      You get      You want
-----      -
BASE 10      +   TASK1 (3)   =   all 13 working
trained      +   new data

```

head grows: 10 boxes -> 13 boxes -> 16 boxes

What goes wrong. Task1's data contains *only* ApplyEyeMakeup, ApplyLipstick and Archery. So the model learns 'everything is one of these three'. It never sees a Diving clip again, so it forgets Diving.

```
Base accuracy:  98.83%  ->  50.29%  ->  19.30%
                start    +Task1     +Task2
```

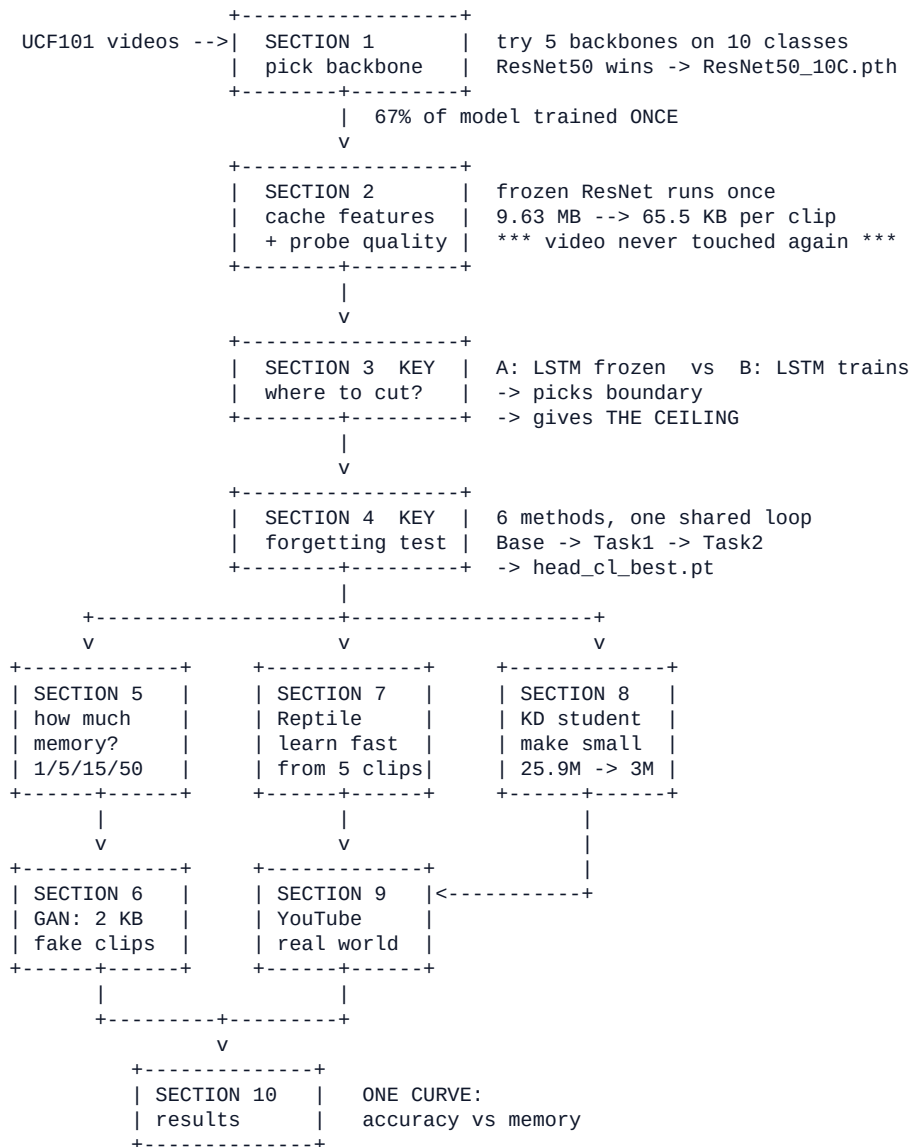
From 99% to 19%. **That is catastrophic forgetting**, and it is what Section 4 is about.

Why Task1 suffers worst

```
BASE      full-body sports      <- far away in feature space
TASK1     ApplyEyeMakeup, ApplyLipstick +
TASK2     BrushingTeeth          +-- all close-up face + hand!
```

ApplyLipstick and BrushingTeeth are essentially the same picture: a face, a hand near the mouth. They fight over the same region. Base is far away and easy to protect. Task1 is not. On top of that Task2 supplies about 100 clips per class while replayed Task1 gets only 15 — a **6.7x vote imbalance**, so the majority simply wins.

Part 4 — The complete schema



The three branches are NOT alternatives. You do all of them. They are drawn side by side because none of them needs the others — they are three different axes (memory, speed, size) measured on the same trained model. They all merge back into Section 10.

Simple order that always works: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10. Every dependency is satisfied by the time you reach it.

Part 5 — The six continual-learning methods

Every method runs through **one shared function** (`run_cl_stream` in `src/cl/trainer.py`) with the same learning rate, epochs, optimiser and schedule. The mechanism flag is the only difference. That is what makes the comparison valid.

Notebook	Method	Old-class signal	Memory/class
4.1	Naive	none	0
4.2	WeightAlign	logit rescale	0
4.3	EWC	weight anchor	Fisher + theta*
4.4	LwF	soft targets, every sample	0
4.5	SmartReplay	hard labels, 15 exemplars	~2 MB
4.6	Replay + LwF	both (iCaRL)	~2 MB

4.1 Naive — do nothing

The lower bound. Base collapses from 98.8% to 19.3%. If this does *not* collapse, the protocol is broken and no other number means anything.

4.2 WeightAlign — turn down the volume (free)

The new classes simply produce *larger* scores, so they win the argmax regardless of the features. Rescale the new-class classifier rows so their average magnitude matches the old ones. Costs zero bytes.

This is the honest baseline for replay to beat. If SmartReplay only matches WeightAlign, then ~2 MB per class bought nothing that a free rescale does not give.

4.3 EWC — mark the important weights

$$\text{loss} = \text{CE} + \lambda \sum_i F_i * (\theta_i - \theta_i^*)^2$$

F_i = how much weight i mattered for the old classes (Fisher information)

Fisher is computed on the **current task only**, at the end of each task — real EWC never looks at old data again. Tune λ from the printed CE/EWC split: **aim for EWC at 5-20% of CE**. At $\lambda=5000$ the penalty was 0.5% of CE and EWC was indistinguishable from Naive; $\lambda=1$ gave about 6%.

4.4 LwF — keep a photocopy of yourself (free)

$$\text{loss} = \text{CE} + \lambda * \text{KL}(\text{student_old_logits} || \text{teacher_old_logits}) \text{ at temperature } T$$

Before each task, freeze a deep copy of the model. While training, force the new model's old-class answers to stay close to what that copy predicts. **The teacher is the model itself**, frozen one task ago. It must be re-snapshotted every task — reusing a stale teacher silently makes the term useless. Only old-class logits are distilled, since the teacher's head has no rows for the new classes.

4.5 SmartReplay — keep some old clips (paid)

Store 15 clips per class and mix them into every batch. Strategy 'hard' keeps the samples *farthest* from each class prototype — boundary cases teach more than typical ones. 'diverse' (greedy coresets) and 'random' are also available and ablated in Section 5.

4.6 Replay + LwF — both (iCaRL)

Same memory as 4.5; the LwF half is free. This is essentially iCaRL (Rebuffi et al., CVPR 2017), and the same recipe underlies LUCIR, PODNet and BiC. The question it answers: **does adding a free mechanism on top of a paid one still help?**

Part 6 — Sections 5 to 10: what comes next

#	In simple words	Example / output
5	Try buffer sizes: 1, 5, 15, 50 exemplars x 3 strategies	Is 50 much better than 5? The accuracy-vs-memory CURVE -- the main result
6	GAN: do not keep clips, DRAW fake ones	Keep only 'average Diving + how much it varies' = 2 KB/class
7	Reptile: learn to learn fast	Show 5 clips of a new class. Normal head 40%, Reptile head 70%
8	Knowledge distillation: make it small	Teacher 25.9M -> student ~3M, keep most of the accuracy
9	YouTube: real messy video	Label only the 10 hardest clips per class, then adapt
10	One table, one figure	accuracy vs memory, log x-axis, every method as one dot

The one line that is the whole project

144 MB/class -> ~2 MB -> 15 KB -> 2 KB
raw clips Sec 4/5 vectors GAN

"How cheap can memory get before the model gets stupid?"

Part 7 — Every question you asked, answered

Q1. Does the GAN part make sense in this pipeline?

Split verdict. The *generative replay* half fits very well and is arguably the strongest novel contribution. The *zero-shot* half does not and cannot work at this scale.

Why the replay half belongs

Your task tracker already asks for a buffer that stores 'per-class prototype (μ , σ) instead of raw clips' — that is exactly what GILReplayBuffer is. It becomes the cheapest point on the memory curve: **2 KB per class with zero clips stored**, versus 2 MB for SmartReplay. Same metrics, same axis, direct comparison.

Why the zero-shot half does not

The CVAE learns a text-to-prototype mapping from **10 (name, prototype) pairs** and must then extrapolate to 10 unseen names. Learning a semantic-to-visual regressor *is* the entire zero-shot problem; the paper uses all 101 UCF101 classes. At 10 base classes it cannot produce a defensible number.

Two bugs that must be fixed first

1. The discriminator never sees a real feature. The code passes μ_b — the class *mean* — as the real sample. That makes the entire real distribution **10 vectors**, and the optimal generator collapses to $\hat{x} = \mu$, ignoring both z and σ . Real GIL feeds per-clip features as real and uses (μ , σ) only as generator conditioning.

2. The t-SNE metric is backwards. The notebook reports 'lower intra-class distance = better generative model'. With bug 1 that is guaranteed true and measures **mode collapse**. After the fix, the honest metric is whether synthetic spread *matches* real spread, not whether it is tighter.

Recommendation: do not delete it — halve it. Keep the generative replay, fix the discriminator, drop the zero-shot half.

Q2. Does this make sense as a project? Do we train just the head or all of it?

I traced every optimiser in the codebase. The teacher's backbone trains **exactly once**, on Task 0. After that, every stage trains only a small head.

```
Teacher (ResNet50 + LSTM) : 25,871,946 params -- frozen after Task 0
EmbeddingHead (16 classes) : 70,416 params -- 0.27% of the teacher
                                teacher is ~367x larger
```

As 'latent-space lifelong learning over a frozen video representation' -- yes

This is a real, current research setting. It has a name: *latent replay*, and it is the same family as L2P, DualPrompt and SimpleCIL. SimpleCIL showed that a frozen backbone plus a simple classifier beats many sophisticated CL methods. **Freezing is not a flaw.**

As 'continual learning for video action recognition' -- no, not unqualified

You never adapt the representation, so you are not doing CL on a video model — you are doing CL on a classifier. That is a scoping problem, not a fatal one, but you must say the scoped version because a reviewer will find it in thirty seconds.

Three genuine strengths

1. More controlled than most CL papers. One fixed representation means every difference between methods is attributable to the method. Papers that train the backbone cannot cleanly separate 'method helped' from 'backbone drifted'.

2. The memory argument is strong and quantified. Latent replay is ~9,400x cheaper than clip replay; the GAN prototype is ~72,000x cheaper.

3. Cheap enough for real science. Extract once, then run five methods, three seeds and a memory sweep in minutes.

Three honest weaknesses

1. The representation ceiling — see Q7. **2. Forgetting in a linear head is the easy kind**, dominated by logit drift, which is why the WeightAlign baseline matters. **3. Domain adaptation has a hard ceiling**: a frozen backbone cannot adapt to low-level shift such as compression or lighting.

Use this wording: 'This project studies continual learning of a video action recogniser under a moving freeze boundary, and measures the accuracy/memory trade-off across five orders of magnitude of replay budget.'

Q3. Is it better with cached embeddings or with real frames?

Embeddings — and not only because it is cheaper. Real frames would probably make your continual learning worse.

More trainable parameters means MORE forgetting

Forgetting is caused by weights moving. Right now 2.4M move. With real frames about 17M would move, layer4 would drift toward makeup features, and the collapse would be worse. **The frozen backbone acts as a regulariser** — that is a feature, not a limitation.

Question	Winner
Highest JOINT accuracy (all classes at once)	frames, clearly
Highest CONTINUAL accuracy after 3 tasks	embeddings, likely
Cost	embeddings, by ~40x

The augmentation argument does not apply to your code

The real cost of caching is losing per-epoch data augmentation. But your preprocessing uses Resize -> CenterCrop -> ToTensor -> Normalize: **deterministic, no random crop, no flip**. And temporal_sample picks its random start *once*, then bakes it into the .pt file. So you currently have zero augmentation anyway — switching to frames buys nothing unless you also rewrite preprocessing.

The time cost is what decides it

	Embeddings	Real frames
One epoch, base task	~2 s	~80 s
One CL arm (15 ep x 2 tasks)	~1 min	~40 min
Section 4 (6 arms)	~7 min	~4 hours
Section 5 (12 configs)	~15 min	~12 hours

Recommendation: stay on embeddings, but run *one* frames control — the joint setting only, full fine-tuning, about 30 minutes. Then you can write: 'Full fine-tuning reaches X% jointly. Our frozen-backbone approach reaches Y% jointly and Z% continually, at 1/40th the training cost and 147x less storage.'

Q4. In the schema, sections 5, 7 and 8 are drawn in parallel. Do I choose one?

No. You do all of them. My original diagram was misleading — parallel meant 'these do not depend on each other', but it read like a menu.

All three branch out of Section 4 because they all start from the same trained head, and they all merge back into Section 10. They are **three different axes**:

Branch	Question it asks	Axis
5 -> 6	how cheap can memory get?	MEMORY
7	how fast can it learn a new class?	SPEED
8	how small can the model get?	SIZE

The arrows that *do* connect are real dependencies: 6 needs 5 (GAN is the cheapest dot on the memory curve); 9 needs both 7 and 8 (YouTube gives only 10 clips per class, which is few-shot, so it uses Reptile; and the student is the thing you would actually deploy).

If short on time: Sections 1, 2, 3, 4 and 10 are the must-have core. Then **Section 5 is the one to protect** — everything else adds dots, but Section 5 draws the line they sit on.

Q5. Why does Task1 forget so much after Task2, even with EWC and replay?

Task1 is not being forgotten in general — it is being eaten by Task2 specifically. Your own logs show it:

```
EWC, during Task2:
ep 01 | val Base=83% Task1=84% Task2=0%
ep 05 | val Base=78% Task1=12% Task2=87%    <- Task1 died as Task2 rose
                                             Base HELD at 78%
```

If this were general forgetting, Base would fall too. Base holds and Task1 collapses: a direct trade with Task2.

Reason 1: the classes overlap

ApplyEyeMakeup, ApplyLipstick (Task1) and BrushingTeeth (Task2) are the same picture — close-up face, hand near the mouth. They occupy the same corner of feature space and fight over it. Base is full-body sport, far away, so nothing threatens it. That is why replay and EWC protect Base easily and cannot protect Task1.

Reason 2: the replay is unbalanced

```
new Task2 data : 301 clips / 3 classes = ~100 per class
buffer replay   : 195 clips / 13 classes = 15 per class

-> Task2 gets 6.7x more samples per class than replayed Task1.
```

For classes that are far apart, imbalance does not matter. For classes fighting over the same boundary, **the majority simply wins**: BrushingTeeth has 100 votes, ApplyLipstick has 15.

How to confirm and how to fix

Confirm: print the confusion matrix on Task1's test set after Task2. Prediction: most ApplyEyeMakeup and ApplyLipstick clips are predicted as BrushingTeeth. The Section 2 'closest pairs' diagnostic predicts this *before* any training.

Fix: raise the replay budget from 15 to 50 (that is literally Section 5); add a balanced fine-tune pass after each task (what iCaRL and BiC do); or weight the loss by inverse class frequency.

Q6. Does it make sense to add knowledge distillation inside the rehearsal training, passing the same model as teacher?

Yes — and your instinct about 'the same model' is exactly right. The teacher *is* the same model, frozen as a snapshot before the current task starts. That is LwF.

Why replay and KD are complementary, not redundant

	Signal	Applies to
Replay	HARD labels: 'this clip is ApplyLipstick'	only the 15 stored clips/class
LwF	SOFT targets: '70% BrushingTeeth, 15% ApplyLipstick, 15% ApplyEyeMakeup'	EVERY ApplyEyeMakeup clip during all ~300 new-task clips

Replay tells you **what** the old classes are. LwF tells you **how the old classes relate to each other** — the boundary between them. Soft targets carry inter-class similarity ('dark knowledge') that hard labels throw away entirely.

Why it targets your exact failure

When the model sees a BrushingTeeth clip, the frozen teacher says 'this is somewhat ApplyLipstick-ish'. That explicitly preserves the **decision boundary** between the competing classes. Replaying 15 ApplyLipstick clips with hard labels does not do that — it only says 'these 15 are ApplyLipstick' and says nothing about where the boundary sits. This is not a generic addition; it attacks the observed failure mode.

Three correctness details

1. Re-snapshot the teacher after every task. The teacher for Task2 is the model after Task1, not the original base model. Getting this wrong quietly makes it useless.
2. Distill only the old-class logits. The teacher's head has no rows for new classes.
3. Apply KD to the whole batch (new + replayed), not just replayed samples. That is where the extra signal comes from.

Replay + distillation together is essentially **iCaRL** (Rebuffi et al., CVPR 2017). You would be implementing a well-established method, not inventing one.

Implemented. `train_cl_arm` now takes `teacher`, `kd_lambda` and `kd_T`. Notebooks 4.4 (LwF alone, 0 bytes) and 4.6 (Replay + LwF) were added.

Q7. The ResNet was fine-tuned on 10 classes. Will the embeddings it extracts for NEW tasks be wrong? Is that a problem?

Yes, it is a real limitation — but your own run already proves it is survivable, and Section 3 exists precisely to measure how big it is.

Your data already answers part of it

EWC	Task1 = 98.11%	right after learning it
SmartReplay	Task1 = 96.23%	right after learning it

Those are **frozen** ResNet embeddings of classes the ResNet never saw. If the features could not represent them, you could never reach 96-98%. So empirically the frozen features carry the new classes fine. What you lost afterwards was *forgetting*, not *representation*.

Three reasons it is less bad than intuition says

1. Only layer4 was fine-tuned. conv1 through layer3 (8.5M parameters) still hold generic ImageNet features — edges, textures, shapes. Those are the same for diving and lipstick.

2. The output is 2048-d, and cross-entropy only constrained about 9 directions. To separate 10 classes the loss only needs linear separability, which pins down roughly 9 directions out of 2048. The other ~2039 are unconstrained by the loss and keep their ImageNet structure. That is exactly why 'frozen backbone + linear probe' works so well.

3. The real bottleneck is the LSTM, not the ResNet.

ResNet output	2048 numbers	<- lots of room, mildly specialised
LSTM output	256 numbers	<- squeezed to EXACTLY what 10 sports classes need

The 256-d summary is where information genuinely got thrown away. **That is the whole reason we cache at 2048 rather than 256.**

Section 3 is literally the measurement of this question

Probe	The question it asks
A (LSTM frozen)	can the 10-sports-class 256-d summary hold new classes?
B (LSTM trains)	can we re-learn a summary from the 2048-d features?

B much greater than A means the summary was too specialised but the 2048-d features were fine — the problem exists and keeping the LSTM trainable solves it. **B about equal to A** means there was no problem. Either way you have *measured* it instead of assuming. Your question is not a gap in the design — it is the design.

And the ceiling closes the loop

Joint training uses the **same frozen features**. If it reaches 85%, the features provably can hold all 16 classes, so any CL arm below 85% is losing it to forgetting. The ceiling separates the two failure modes; without it you could not tell them apart.

Where it genuinely bites

1. Fine-grained pairs the base task never needed — BoxingPunchingBag versus BoxingSpeedBag differ by a small object. If layer4 discarded that detail, no amount of LSTM retraining recovers it. **2.** Low-level domain shift in Section 9: frozen conv layers cannot adapt to YouTube compression or lighting. State that as a limitation.

If the ceiling comes back low (say 60%), that is the signal the representation is the bottleneck. The fix is more diverse base classes — uncomment the 14 commented-out classes in config.py so the base is not only sports.

An optional experiment that would sharpen this

A subtler version of the worry: did fine-tuning on 10 sports classes make the features *worse* for makeup than plain ImageNet features would be? Add a **Probe C**: extract features with the original ImageNet ResNet50 (no Task-0 fine-tune) and run the same joint training. If C beats A and B, fine-tuning on 10 narrow classes actively hurt transfer. One extra extraction pass, and it would make a strong paragraph.

Part 8 — Repository layout and run order

```
notebooks/
  ULAL.ipynb          <- principal: winners only, written LAST
  study/
    1_choose_model/   DONE    1.0_backbone_transfer.ipynb
    2_extract_features/ BUILT  2.0_cache_frame_features.ipynb (+ probes)
    3_freeze_study/    BUILT  3.0_freeze_boundary.ipynb
    4_continual_learning/ BUILT 4.0_cl_setup.ipynb    <- run first
                                4.1_naive.ipynb
                                4.2_weight_align.ipynb
                                4.3_ewc.ipynb
                                4.4_lwf.ipynb
                                4.5_smart_replay.ipynb
                                4.6_replay_lwf.ipynb
                                4.7_compare.ipynb
    5_memory_ablation/ ... 10_results/
    _archive/          superseded notebooks, kept for history

src/
  bootstrap.py        clone + dataset + paths (replaces both bash scripts)
  imports.py          loads everything into the notebook namespace
  config/config.py     classes, paths, hyperparameters (env-var driven)
  data/cache.py        feature cache, label space, verification
  cl/trainer.py        train_cl_arm, run_cl_stream -- ONE loop for every arm
  models/temporal_head.py TemporalHead, make_temporal_head, expand_head, weight_align
  utils/probe.py       embedding-quality probes and geometry
  gil/gil_gan.py       GAN components (Section 6)
```

Run order

```
2.0 cache features + probe  ~30 min once, then ~30 s to load
3.0 freeze study           -> gives FREEZE_LSTM and the CEILING
4.0 cl setup               -> base head + shared protocol
4.1 ... 4.6               any order, each saves its own JSON
4.7 compare                reads whatever has run
```

Design rule that makes the split notebooks valid

4.0 writes head_base.pt and stage1_protocol.json. Every arm loads both and goes through the same run_cl_stream, so learning rate, epochs, batch size and schedule are identical **by construction** even though the arms run in separate notebooks.

Always push before running. The notebooks clone src/ from GitHub, so uncommitted local changes are invisible to Colab. The bootstrap cell now verifies the clone contains the key files and tells you to push if not.

Part 9 — Bugs found and fixed

Protocol and correctness

Bug	Impact	Fix
Best-val epoch selection in CL	Stopped Naive at epoch 2 -- before it forgot anything	One trained task (Base 9%, Task 1 43%) -- while EV
EWC lambda 100x too weak	CE 0.38 + EWC 0.0019 -- the penalty was 0.5% of the loss from EWC	Loss from EWC a SEW split; aim 5-20%
Fisher key prefix	compute_fisher runs on ModelWrapper, prefixing keys	Is with original model trainer; input mapping, the penalty mat
extract_class_features swept	The GAN notebook's 'seen accuracy' was really TRAP accuracy	Penalty extraction via extract_frame_features

Structural

Bug	Impact	Fix
src/meta_learning/ loaded AF	21 src models collisions. Editing src/models/teacher.py had NO effect	Pytest NO effecting any notebook
config.py executed twice	config.config and src.config.config were two module	Single import two copies of every value; mutating one was i
setup_colab.sh sed-patched	Working tree diverged from git; any re-clone silently re	Added your pathen. Also added file and resolved
BRANCH='GAN&ActiveLearning'	It in a bash and backgrounds the command in a shell subprocess with a LIST, no shell	
fetch_src.sh	Had to already exist on Drive (chicken-and-egg); one	One shallow file clone, self-contained cell
SmartReplayBuffer .norm(dim=C)	Crashed on (N, T, D) sequence exemplars	flatten(1) -- shape-agnostic
finetune_head computed MSE	Forced it to fabricate a dummy tensor; crashed	train_epoch skips the term entirely

Still open

Issue	Fix in
TASK_1 / TASK_2 have 7 of 10 classes commented out (16 classes, not 50)	config.py
GAN discriminator sees mu_b as 'real' -- generator collapses	Section 6
cfg.GIL_cfg.GIL_CKPT_DIR -- AttributeError	Section 6
avg_intra_dist called with numpy arrays -- AttributeError	Section 6
TSNE(n_iter=) removed in scikit-learn 1.7	Section 6
Student LSTM randomly initialised and never trained -- KD distils nothing	Section 8

The single highest-value next step: run Section 2 and read the probe table, then Section 3 and read the A-vs-B gap. Those two numbers determine whether the project rests on solid ground — and both are cheap.